

2025

Proje Raporu

Proje Adı: En Düşük Maliyetle Yol Optimizasyonu
Geliştirici Adı: Dilara ERDOĞAN

1. Klasik Programlama Metodu(Tek İş Parçacıklı)

```
public class SingleThread {

    static final int yuzMetreIcinYolMaliyeti = 20000;

    static final int yuzMetreIcinKopruMaliyeti = 40000;

    static final int nehrinGenisligi = 600;

    static final int yatayMesafe = 800;

    static final int maxMaliyet=400000;

    public static double maliyetHesapla(double x) {

        double kopruUzunlugu=Math.sqrt(nehrinGenisligi*nehrinGenisligi + x*x);

        double yolUzunlugu=yatayMesafe-x;

        double toplamMaliyet=(kopruUzunlugu/100)*yuzMetreIcinKopruMaliyeti+
(yolUzunlugu/100)*yuzMetreIcinYolMaliyeti;

        return toplamMaliyet;

    }

    public static void main(String[] args) {

        int x;

        double minMaliyet=maxMaliyet;

        int enAzMaliyetGetirenYatayUzaklik=0;

        for(x=1; x<=yatayMesafe; x++) {

            double sonuc = maliyetHesapla(x);

            if(sonuc<minMaliyet) {

                minMaliyet=sonuc;

                enAzMaliyetGetirenYatayUzaklik=x;

            }

        }

    }

}
```

```

        else {
            break;
        }
    }
}

```

```

        double kopruUzunlugu=Math.sqrt(nehriGenisligi*nehriGenisligi +
enAzMaliyetGetirenYatayUzaklik*enAzMaliyetGetirenYatayUzaklik);

```

```

        double yolUzunlugu=yatayMesafe-enAzMaliyetGetirenYatayUzaklik;

```

```

        System.out.println("-En dusuk maliyetli durum-");

```

```

        System.out.println("Toplam yol uzunlugu: "+yolUzunlugu);

```

```

        System.out.println("Toplam kopru uzunlugu: "+kopruUzunlugu);

```

```

        System.out.println("Toplam Maliyet: "+minMaliyet+" dolar");

```

```

        XYSeries yolMaliyet=new XYSeries("Toplam Maliyet (dolar)");

```

```

        for(int i=0; i<=yatayMesafe; i+=1) {
            yolMaliyet.add(i, maliyetHesapla(i));
        }

```

```

        XYSeries optimumNokta = new XYSeries("Min Maliyet (Optimum Nokta) (" +minMaliyet+"
$)");

```

```

        optimumNokta.add(enAzMaliyetGetirenYatayUzaklik, minMaliyet);

```

```

        XYSeriesCollection grafikBilgisi=new XYSeriesCollection();

```

```

        grafikBilgisi.addSeries(yolMaliyet);

```

```

        grafikBilgisi.addSeries(optimumNokta);

```

```

        JFreeChart grafik=ChartFactory.createXYLineChart("Köprü Konumuna Göre Toplam
Maliyet", "Köprü Konumu (metre)", "Toplam Maliyet (dolar)", grafikBilgisi);

```

```

        XYPlot plot = grafik.getXYPlot();

```

```

        plot.getRangeAxis().setRange(350000, plot.getRangeAxis().getUpperBound());

```

```

XYLineAndShapeRenderer render = new XYLineAndShapeRenderer();

render.setSeriesPaint(0, Color.BLUE);

render.setSeriesShapesVisible(0, false);

render.setSeriesPaint(1, Color.RED);

render.setSeriesShapesVisible(1, true);

render.setSeriesShape(1, new Ellipse2D.Double(-5, -5, 10, 10));


plot.setRenderer(render);


JFrame grafikGoruntusu=new JFrame("Maliyet Grafiği");

grafikGoruntusu.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

grafikGoruntusu.setSize(600, 500);

grafikGoruntusu.setLocationRelativeTo(null);

grafikGoruntusu.add(new ChartPanel(grafik));

grafikGoruntusu.setVisible(true);

}

}

```

2. Thread Kullanarak Çözüm(Multi Thread)

```

public class MultiThread {

static final int yuzMetrelcinYolMaliyeti = 20000;

static final int yuzMetrelcinKopruMaliyeti = 40000;

static final int nehrinGenisligi = 600;

static final int yatayMesafe = 800;

static final int maxMaliyet = 400000;

static final int threadSayisi = 4;

```

```

static class maliyetHesaplayici extends Thread {

    private int baslangic, bitis;

    private double minMaliyet = maxMaliyet;

    private int enAzMaliyetGetirenYatayUzaklik;


    public maliyetHesaplayici(int baslangic, int bitis) {

        this.baslangic = baslangic;

        this.bitis = bitis;

    }


    @Override

    public void run() {

        for (int x = baslangic; x <= bitis; x++) {

            double sonuc = maliyetHesapla(x);

            if (sonuc < minMaliyet) {

                minMaliyet = sonuc;

                enAzMaliyetGetirenYatayUzaklik = x;

            } else {

                break;

            }

        }

    }


    public double getMinMaliyet() {

        return minMaliyet;

    }


    public int getEnAzMaliyetGetirenYatayUzaklik() {

        return enAzMaliyetGetirenYatayUzaklik;

    }

}

```

```
}  
}
```

```
public static double maliyetHesapla(double x) {  
  
    double kopruUzunlugu = Math.sqrt(nehirinGenisligi * nehirinGenisligi + x * x);  
  
    double yolUzunlugu = yatayMesafe - x;  
  
    return (kopruUzunlugu / 100) * yuzMetrelcinKopruMaliyeti + (yolUzunlugu / 100) *  
yuzMetrelcinYolMaliyeti;  
  
}
```

```
public static void main(String[] args) {  
  
    int aralik = yatayMesafe / threadSayisi;  
  
    maliyetHesaplayici[] threadler = new maliyetHesaplayici[threadSayisi];  
  
    for (int i = 0; i < threadSayisi; i++) {  
  
        int baslangic = i * aralik + 1;  
  
        int bitis = (i == threadSayisi - 1) ? yatayMesafe : (i + 1) * aralik;  
  
        threadler[i] = new maliyetHesaplayici(baslangic, bitis);  
  
        threadler[i].start();  
  
    }
```

```
  
    double minMaliyet = maxMaliyet;  
  
    int enAzMaliyetGetirenYatayUzaklik = 0;  
  
    for (maliyetHesaplayici thread : threadler) {  
  
        try {  
  
            thread.join();  
  
            if (thread.getMinMaliyet() < minMaliyet) {  
  
                minMaliyet = thread.getMinMaliyet();  
  
            }  
  
        }  
  
    }
```

```
        enAzMaliyetGetirenYatayUzaklik = thread.getEnAzMaliyetGetirenYatayUzaklik();  
    }  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
}
```

```
double kopruUzunlugu = Math.sqrt(nehirinGenisligi * nehirinGenisligi +  
enAzMaliyetGetirenYatayUzaklik * enAzMaliyetGetirenYatayUzaklik);
```

```
double yolUzunlugu = yatayMesafe - enAzMaliyetGetirenYatayUzaklik;
```

```
System.out.println("-En dusuk maliyetli durum-");
```

```
System.out.println("Toplam yol uzunlugu: " + yolUzunlugu);
```

```
System.out.println("Toplam kopru uzunlugu: " + kopruUzunlugu);
```

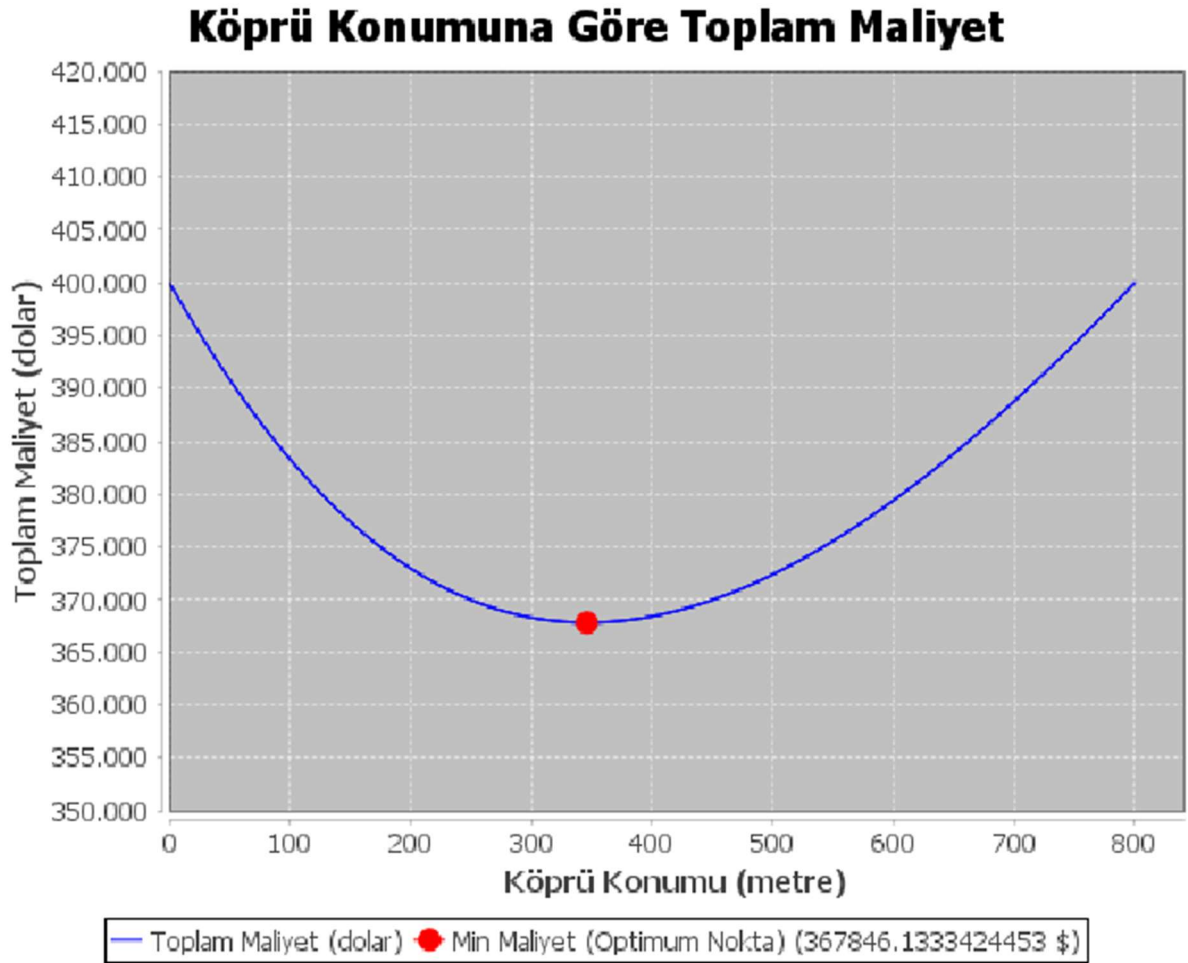
```
System.out.println("Toplam Maliyet: " + minMaliyet + " dolar");
```

NOT: Kod tekrarını önleme amacıyla grafik çizdirme kodu single thread içinde yer verilmiştir.

PROJE HAKKINDA VE GRAFİK YORUMLAMA

Bu ödevde oluşturulan senaryoya göre fabrika ve depo arasındaki köprü ve yol bağlantısı için en az maliyetli durum hesaplandı. Bu hesaplamada Öklid bağıntısı ve küçük-büyük karşılaştırılması kullanıldı. En az maliyetli durumun (optimum) köprü uzunluğu, yol uzunluğu ve toplam maliyeti ekrana çıktı olarak yazdırıldı. Grafik bilgisinde optimum nokta gösterildi.

Ödev için java programlama dili tercih edildi ve Eclipse IDE programında yazıldı. Grafik çizimi için JFreeChart kütüphanesi kullanıldı.



Grafikte görüldüğü gibi kırmızı nokta optimum noktadır. En az maliyet toplam 367846.13 \$'dır. Yatay mesafenin yarısı için düşündüğümüzde x arttığında maliyetin azaldığını görmekteyiz. Bu durum x'in optimum noktaya geldiğini ve o noktada köprü uzunluğu ve yol uzunluğunun birbirini dengeleyerek en az maliyeti verdiğini gösterir. Yatay mesafenin kalan yarısı için x arttığında köprü uzunluğu daha çok artacak ve fazla maliyet getirecek, yol ise daha çok kılalacak ve az maliyet getirecek. Bu durum toplam maliyeti tekrar arttıracaktır. Grafiğin parabolik görünmesinin sebebi budur.